

Goal

My original goal for this project was to compare a Long-Short Term Memory neural network architecture to a Transformer architecture in forecasting GDP. To begin with, I wanted to use the Federal Reserve's FRED database; however, I quickly realized that this wouldn't be enough data to extract anything meaningful, as there were about 113 quarters available, leading to a ~1:500 data to parameter ratio for the models.

Data Pipeline

Eventually, I found the Organization for Economic Co-operation and Development's OECD SDMX API that has a large database on OECD countries. I decided on five indicators to track (Composite leading indicator, unemployment rate, CPI, long-term interest rates, industrial production) and also recorded the GDP. I had to explicitly exclude some non-country groups that the OECD tracks such as the OECD as a whole, G20, and EA19. Due to few countries having CLI, I didn't want to cut out even more data, so I decided to put CLI_level and CLI_diff to be 0 for the country-quarters that didn't have it and added a parameter CLI_missing that told the model whether we had access to this data. This moved the number of countries that were covered by the data from 9 to 29. GDP growth is also included as a lagged feature of the model. The final data includes 3,924 quarters across 29 countries with 10 feature columns.

Windowing & Split

make_windows creates 8-quarter lookback sliding per-country windows. The train/validation/test split was made with fixed dates that are the same for each country. This was done to test temporal generalization (whether the past quarters can help predict the future) rather than cross-country generalization. This resulted in 3,254 train / 216 val / 222 test windows.

Architectures

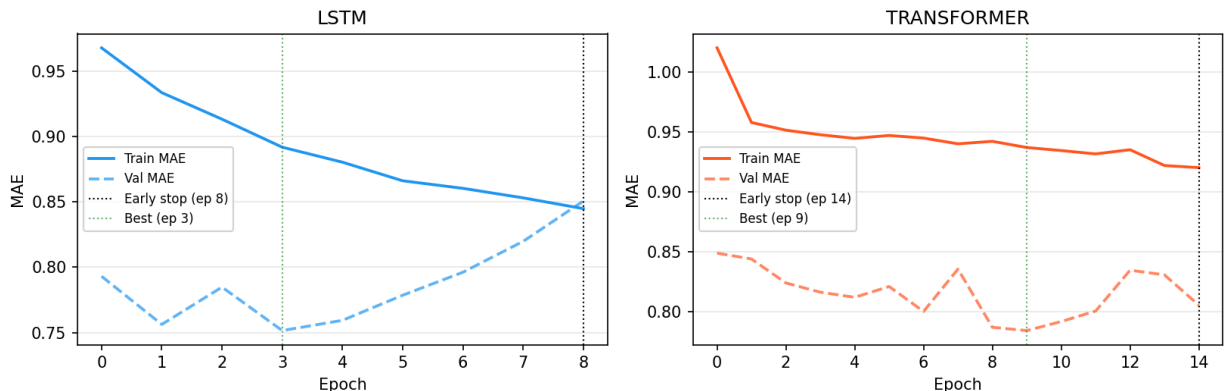
For both models the countries are embedded into four dimensional vectors and then concatenated to the models per-timestep feature column in forward(). LSTM: nn.LSTM(input_dim+embedding_dim, hidden_dim, layer_dim, batch_first=True), country embedding repeated across all 8 timesteps and concatenated before the recurrent layers, final prediction from the last timestep's hidden state. Transformer: causal (masked) self-attention specifically chosen over bidirectional, to make the comparison structurally fairer to the LSTM's strictly-left-to-right processing; uses a learned positional embedding added per-timestep; nn.TransformerEncoderLayer + nn.TransformerEncoder stack; causal mask via generate_square_subsequent_mask; prediction taken from the last (causally-summarized) position, same logic as the LSTM.

Training Setup

I chose to share the same training loop for both models in order to make the comparison fair, meaning they use the same loss, optimizer, and batching. I switched the loss function from MSE

to L1/MAE because MSE's squared-error penalty let a few high-error countries dominate the gradient, since squaring exaggerates already-large errors disproportionately. IndicatorDataset and DataLoader bundle my X, y, countries_idx, and groups (which has the country names in it) together so that shuffling doesn't desynchronize them. I implemented early stopping after finding that there was some variation in where the model's best epoch, but it was generally in the 4-15 range depending on the architecture.

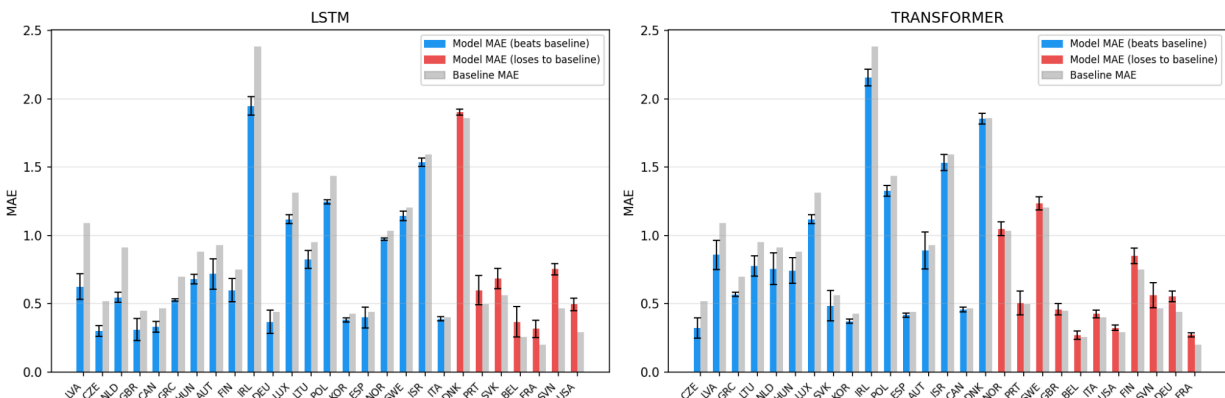
Train vs Validation Loss — seed 0



Evaluation

I used the test set exactly once at the end after making my adjustments to the hyperparameters based on the validation set. I used naive-mean baseline comparison to evaluate my model, which attempts to predict the next GDP growth based on the mean of the previous GDP growths. Multi-seeding was used to prevent interpretations based on single, noisy runs. Instead, I aggregated data over five seeds, looking at the means +/- the standard deviations per country.

Per-Country Val MAE vs Naive Mean Baseline (5 seeds)

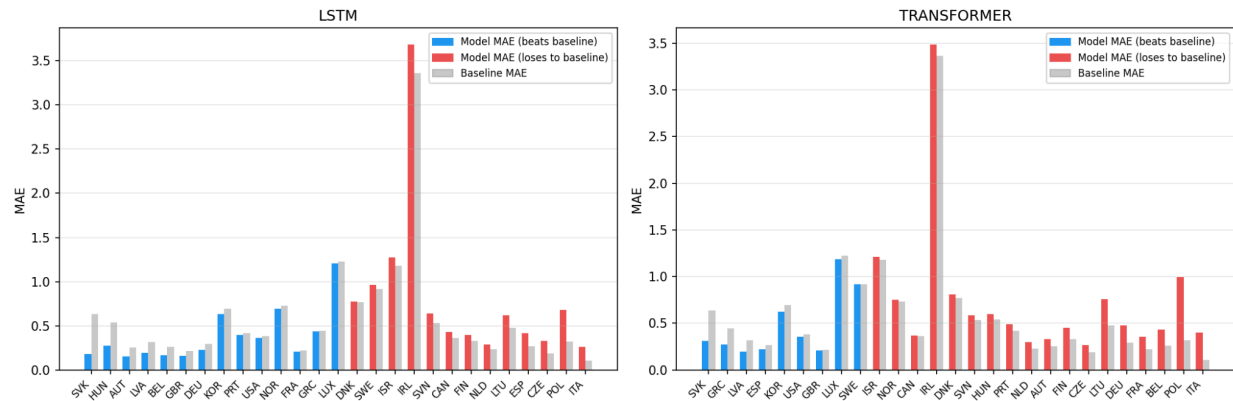


Key Findings

At the final epoch of a 100-epoch run, the LSTM loses to the naive baseline on 22 of 27 countries (some by 6-8x); at its best validation epoch, it beats baseline on 19-20 of 27, consistently across 5 seeds. This is a clear example of overfitting happening over the course of

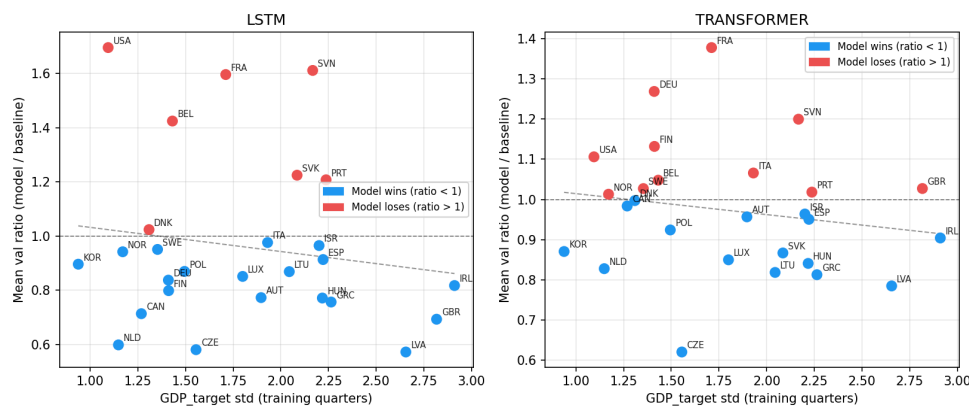
the 100 epochs, which is why early stopping was implemented. The countries for which the models beat the baseline are consistent across the multi-seeding. The test-set evaluation (single run) was broadly consistent with validation: 14 of 27 countries beat baseline.

Per-Country Test MAE vs Naive Mean Baseline (seed 0)



To determine why the model consistently underperforms in some countries I determined the correlation between beats-baseline ratio and GDP-growth volatility (std) which is weak and wrong-signed (-0.145); correlation with training history length is also weak (+0.294). This lack of correlation is demonstrated by the graphs below. Closer inspection suggests two distinct failure modes bundled together: very low-volatility countries (USA) where the baseline is already near-optimal, and very high-volatility countries (SVN, SVK, PRT) where neither baseline nor model tracks the noise well.

Val Ratio (model MAE / baseline) vs GDP Growth Volatility



Limitations

OECD data uses revised values, not first release vintage, so there is some look-ahead bias. Validation and test windows were very small due to the shortage of data, so multi-seeding was used to mitigate the noise, but it still exists.